



AutoHub Pro

Central de Automacoes

Documentacao Tecnica Completa

Este documento descreve a arquitetura, tecnologias, modulos e decisoes de implementacao do projeto AutoHub Pro.

v1.0.0 | Python 3.13 | Windows 10 / 11

Autor: Pedro Henrique Santana Nascimento

Gerado em: 09/06/2026 as 09:16

1. O que é o AutoHub Pro?

AutoHub Pro é uma aplicação desktop para Windows construída com Python 3.13 e Tkinter. Ela funciona como uma central que reúne três automações distintas em uma única interface gráfica com tema escuro inspirado no GitHub Dark. O objetivo é simplificar tarefas repetitivas internas: análise de logs de servidor, download de imagens de portais web e avaliação de acessibilidade de sites.

Modulo	Tecnologia principal	Saida gerada
Analizador de Logs	Paramiko (SSH)	Relatorio .txt com contagem de erros
Scraper de Imagens	Requests + BeautifulSoup	Imagens .jpg + planilha .xlsx
AMAWeb	Selenium (Chrome)	Planilha .xlsx com notas de acesso

2. Estrutura de Pastas

O projeto é organizado em pacotes Python separados por responsabilidade:

```
interface_automacao/  
  app.py          <- ponto de entrada da aplicacao  
  criar_atalho.py <- gera icone .ico e atalho .lnk no Desktop  
  gerar_pdf.py    <- gera o manual de instalacao em PDF  
  requirements.txt <- dependencias pip  
  .env            <- credenciais SSH (nao versionado)  
  .env.example    <- template das variaveis de ambiente  
  
  automations/  
    logs.py       <- analisador de logs via SSH  
    scraper.py    <- downloader de imagens  
    amaweb.py     <- avaliador de acessibilidade  
  
  config/  
    theme.py      <- cores, fontes e constantes visuais  
  
  ui/  
    app.py        <- classe principal da janela (AutoHubApp)  
    utils.py      <- helper de output por fila  
  
  assets/  
    icon.ico      <- icone gerado pelo criar_atalho.py
```

3. Modulo 1: Analisador de Logs

Arquivo: automations/logs.py

Finalidade: Conecta via SSH em servidores internos, le arquivos de log no formato XML (log4j) e gera um relatorio consolidado com contagem de padroes de erro.

Como funciona

- As credenciais SSH (usuario e senha) sao carregadas do arquivo .env via python-dotenv
- A data de analise e selecionada pelo usuario na interface (formato dd/mm/aaaa)
- O modulo conecta nos servidores Elasticsearch (192.168.200.124, .161, .162) e Liferay (.140, .147)
- Os logs sao lidos remotamente via SSH usando o Paramiko, sem copiar os arquivos
- O XML e analisado para contar 16 padroes de erro predefinidos
- Os resultados sao exibidos com barras visuais (caracteres de bloco) no terminal
- Erros de template FTL sao agrupados e exibidos separadamente
- O usuario pode fazer download do relatorio como arquivo .txt

Padroes de erro monitorados

- ElasticsearchStatusException
- 404 Not Found
- Broken Pipe / Connection Reset
- LDAP authentication failures
- FTL template errors
- SAX Security Manager issues
- E outros 10 padroes internos

Tecnologia usada

Biblioteca	Versao	Para que serve
paramiko	3.0.0+	Conexao SSH e execucao de comandos remotos
python-dotenv	1.0.0+	Leitura de credenciais do arquivo .env
xml (stdlib)	nativo	Parse dos arquivos de log no formato XML

4. Modulo 2: Scraper de Imagens

Arquivo: automations/scraper.py

Finalidade: Le uma lista de URLs de um arquivo .txt, acessa cada portal, extrai todas as imagens e baixa apenas as que estao dentro do limite de tamanho.

Como funciona

- Le o arquivo de URLs (uma por linha) - padrao: fontes/links.txt
- Para cada URL: faz requisicao HTTP, extrai todas as tags com BeautifulSoup
- Faz uma requisicao HEAD para verificar o tamanho antes de baixar (economiza banda)
- Filtra: minimo 40 KB e maximo 2 MB - imagens fora desse intervalo sao ignoradas
- Usa ThreadPoolExecutor com 5 workers por URL para downloads simultaneos
- Salva as imagens como JPEG na pasta galeria_noticias/

- Ao final, grava ou atualiza registro_noticias.xlsx com estatísticas da varredura

Colunas do Excel gerado

Coluna	Descricao
Data	Data/hora da execucao
Portal	Nome extraido do caminho da URL
URL	URL original varrida
Encontradas	Total de imagens encontradas na pagina
Baixadas	Imagens dentro do filtro de tamanho, efetivamente salvas
Fora_Padrao	Imagens acima de 2 MB que foram ignoradas

5. Modulo 3: AMAWeb

Arquivo: automations/amaweb.py

Finalidade: Avalia automaticamente a acessibilidade de uma lista de sites acessando o portal AMAWeb (amaweb.unifesp.br) via navegador Chrome headless.

Como funciona

- Le o arquivo de dominios/URLs - padrao: urls.txt
- Inicia o Chrome em modo headless (sem interface grafica) usando Selenium
- Para cada site, navega ate a pagina de resultados do avaliador AMAWeb
- Aguarda ate 180 segundos para o carregamento completo da pagina
- Extrai a nota do seletor CSS .score-circle-value
- Se o site expirar o tempo ou der erro, registra 'Timeout / Erro' e continua
- Exibe progresso em tempo real: nota atual, tempo medio, estimativa restante e barra visual
- Salva os resultados em resultado.xlsx (caminho configuravel pela interface)

Configuracao do Chrome Headless

O Selenium e iniciado com flags de estabilidade e segurança para funcionar sem janela:

```
--headless
--no-sandbox
--disable-dev-shm-usage
--disable-gpu
--window-size=1920,1080
```

6. Tecnologias e Bibliotecas

Todas as dependencias externas estao em requirements.txt e sao instaladas via pip.

Biblioteca	Versao min.	Usado em
tkinter	nativo	Interface grafica - janela, botoes, terminal, sidebar
paramiko	3.0.0	Modulo de Logs - conexao SSH e leitura remota
requests	2.31.0	Scraper - requisicoes HTTP para buscar HTML e imagens
beautifulsoup4	4.12.0	Scraper - parse do HTML para encontrar tags
selenium	4.18.0	AMAWeb - automacao do Chrome (headless)
pandas	2.0.0	Scraper e AMAWeb - criacao e leitura de planilhas
openpyxl	3.1.0	Scraper e AMAWeb - leitura/escrita de arquivos .xlsx
pillow	10.0.0	criar_atalho.py - geracao do icone multi-resolucao
python-dotenv	1.0.0	Logs - carregamento de credenciais do .env
fpdf2	2.x	gerar_pdf.py - geracao dos manuais em PDF
tqdm	4.66.0	Barras de progresso no terminal

7. Interface Grafica (ui/app.py)

A janela principal é implementada na classe `AutoHubApp` em `ui/app.py`. O layout é dividido em quatro regiões fixas:

- Sidebar (215px): navegação com 3 botões + logo do projeto
- Header: título da página atual + badge de status
- Área de conteúdo: cards de configuração por módulo (seletor de arquivo/pasta + botões Run/Stop)
- Terminal de output: widget de texto com cores por tipo de mensagem
- Status bar: relógio em tempo real + indicador da operação atual

Cores do terminal de output

Tag	Cor	Quando e usada
normal	Branco	Mensagens gerais
info	Azul (#58a6ff)	Informações e status
success	Verde	Conclusão bem-sucedida
warning	Amarelo	Avisos e alertas
error	Vermelho	Erros e falhas
progress	Laranja	Atualizações de progresso
dim	Cinza escuro	Texto secundário / separadores
title	Branco negrito	Títulos de seção no output

8. Modelo de Threading

Para que a interface não trave durante a execução das automações, cada módulo roda em uma thread separada. A comunicação com a UI é feita de forma segura via fila (`queue.Queue`).

Fluxo de execução

- Botão Run: cria um `threading.Event` (`stop_ev`) e inicia uma thread com a função do módulo
- A função do módulo recebe a fila (`out_q`) e o evento de parada (`stop_ev`)
- Para enviar texto ao terminal, o módulo chama: `out_q.put((tag, mensagem))`
- A UI faz polling da fila a cada 50ms com `_poll_output()` e renderiza as mensagens
- Botão Stop: chama `stop_ev.set()`, sinalizando a thread para encerrar graciosamente
- Ao terminar, a thread envia `out_q.put(('DONE', None))` para notificar a UI

Por que essa arquitetura?

MOTIVO Tkinter não é thread-safe: nunca atualize widgets diretamente de uma thread secundária. A fila (`Queue`) age como canal seguro - a thread escreve, a UI lê no seu próprio ciclo de 50ms.

```
# Padrão seguido por todos os módulos:
def run_modulo(input_path, output_path, out_q, stop_ev):
    def log(msg, tag='normal'):
        out_q.put((tag, msg + '\n'))
```

```
log('Iniciando...', 'info')
for item in lista:
    if stop_ev.is_set(): # verifica cancelamento
        break
    # ... processar item ...
    log(f'Processado: {item}', 'success')

out_q.put(('DONE', None)) # sinaliza conclusao
```

9. Sistema de Tema (config/theme.py)

Todas as cores, fontes e tamanhos da interface estao centralizados em config/theme.py. Isso garante consistencia visual e facilita mudancas futuras - basta alterar um valor nesse arquivo para refletir em toda a aplicacao.

Paleta de cores (GitHub Dark)

Constante	Hex / RGB	Onde e aplicada
DARK (fundo principal)	#0d1117	Background da janela e paginas
DARK2 (fundo secundario)	#161b22	Cards, header, footer
DARK3 (fundo terciario)	#21262d	Code blocks, linhas de tabela
SIDEBAR	#010409	Painel lateral de navegacao
ACCENT (azul)	#58a6ff	Botoes, destaques, links
GREEN (sucesso)	#3fb950	Status OK, mensagens de sucesso
RED (erro)	#f85149	Erros, alertas criticos
YELLOW (aviso)	#d29922	Avisos, alertas moderados
ORANGE (progresso)	#e3702b	Mensagens de progresso
TEXT (texto principal)	#e6edf3	Todo texto de conteudo
TEXT2 (texto secundario)	#8b949e	Subtitulos, labels, descricoes

Fontes utilizadas

- Interface (UI): Segoe UI - fonte nativa do Windows, clara e moderna
- Terminal de output: Consolas - monospace nativa do Windows, ideal para logs
- Tamanhos: 7px (labels pequenos) ate 14px (titulos de secao)

10. Utilitarios e Scripts Auxiliares

criar_atalho.py

Script que cria um atalho (.lnk) do aplicativo na Area de Trabalho do Windows. Ele tambem gera automaticamente o icone .ico multi-resolucao usando Pillow.

- Gera o icone em 6 resolucoes: 16, 32, 48, 64, 128 e 256 pixels
- O icone tem fundo escuro (#0d1117) com borda de destaque e letra 'A' centralizada
- O atalho usa pythonw.exe para abrir o app sem mostrar janela de console
- O arquivo AutoHub Pro.lnk e criado diretamente na Area de Trabalho

gerar_pdf.py

Script que gera o manual de instalacao do projeto em formato PDF usando fpdf2. Segue o mesmo tema visual da aplicacao (GitHub Dark) com header, footer e estilos proprios.

ui/utlis.py

Helper simples com a funcao de polling da fila de output. Centraliza a logica de leitura da queue e atualizacao do widget de

terminal.

11. Configuracao de Ambiente (.env)

O arquivo .env armazena credenciais sensiveis que NAO devem ser versionadas no git. O arquivo .env.example serve de template e PODE ser versionado.

Variaveis necessarias

```
# .env.example
LIFERAY_USUARIO=seu_usuario
LIFERAY_SENHA=sua_senha
```

Essas credenciais sao usadas exclusivamente pelo modulo Analisador de Logs para autenticar as conexoes SSH nos servidores internos.

SEGURANCA arquivo .env esta listado no .gitignore e nunca deve ser commitado. Cada desenvolvedor deve criar o proprio .env localmente a partir do .env.example.

12. Resumo Tecnico do Projeto

Aspecto	Detalhe
Linguagem	Python 3.13
Interface grafica	Tkinter (nativo do Python)
Sistema operacional	Windows 10 / 11
Versao	v1.0.0
Modulos de automacao	3 (Logs, Scraper, AMAWeb)
Dependencias externas	11 pacotes pip
Modelo de execucao	Multi-thread com Queue para comunicacao UI
Armazenamento	Baseado em arquivos (Excel, TXT, JPEG)
Banco de dados	Nenhum
Rede	SSH (Logs) + HTTP (Scraper) + Selenium (AMAWeb)
Linhas de codigo (est.)	~1.330 linhas no total
Autor	Pedro Henrique Santana Nascimento
Gerado em	09/06/2026 as 09:16

OBSERVACAO Este documento foi gerado automaticamente pelo script gerar_documentacao.py e reflete o estado do projeto na data indicada acima.